

17. 电话号码的字母组合

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Medium	(Not Available)	(Not Available)	-	-	0

- **Tags:** 哈希表, 字符串, 回溯
- **Companies:** (Not Available)

给定一个仅包含数字 2-9 的字符串，返回所有它能表示的字母组合。答案可以按任意顺序返回。

给出数字到字母的映射如下（与电话按键相同）。注意 1 不对应任何字母。



Figure 1: img

示例 1:

输入: digits = "23"

输出: ["ad","ae","af","bd","be","bf","cd","ce","cf"]

示例 2:

输入: `digits = ""`

输出: `[]`

示例 3:

输入: `digits = "2"`

输出: `["a","b","c"]`

提示:

- `0 <= digits.length <= 4`
- `digits[i]` 是范围 `['2', '9']` 的一个数字。

Discussion | Solution

解决方法

1. 问题理解

目标: 根据电话按键数字 (2-9) 到字母的映射, 找出给定数字字符串能表示的所有字母组合。输入: 一个仅包含数字 '2' 到 '9' 的字符串 `digits`。输出: 一个包含所有可能字母组合的列表。映射关系: - '2': "abc" - '3': "def" - '4': "ghi" - '5': "jkl" - '6': "mno" - '7': "pqrs" - '8': "tuv" - '9': "wxyz"

2. 思路分析

这又是一个典型的回溯问题。我们可以将生成组合的过程看作是在一个决策树上进行搜索。

决策树模型: - 树的深度等于输入数字字符串 `digits` 的长度。- 树的第 `k` 层代表 `digits` 中的第 `k` 个数字。- 第 `k` 层的每个节点代表该数字对应的字母中的一个。- 从根节点到叶子节点的路径构成一个完整的字母组合。

回溯过程:

1. 映射表: 首先, 需要一个数据结构 (如哈希表或字典) 存储数字到字母字符串的映射。
2. 路径 (**Path**): 记录当前已经选择的字母构成的部分组合 (通常用字符串或列表表示)。
3. 选择列表 (**Choices**): 当前层级 (对应 `digits` 中的某个数字) 可以选择的字母。
4. 结束条件 (**End Condition**): 当路径的长度等于 `digits` 的长度时, 说明找到了一个完整的组合, 将其加入结果列表。

具体步骤:

1. 处理空输入: 如果 `digits` 为空, 直接返回空列表 `[]`。
2. 创建映射表: `mapping = {'2': "abc", '3': "def", ...}`。
3. 定义回溯函数 `backtrack(index, current_combination)`:
 - `index`: 当前正在处理 `digits` 字符串中的哪个数字的索引。
 - `current_combination`: 当前构建的部分字母组合 (字符串)。
4. 递归终止条件:
 - 如果 `index == len(digits)`, 说明已经处理完所有数字, `current_combination` 是一个完整的组合。
 - 将 `current_combination` 加入结果列表 `result`。
 - 返回。
5. 获取当前数字和对应的字母:
 - 获取当前数字 `digit = digits[index]`。
 - 从映射表中获取该数字对应的字母字符串 `letters = mapping[digit]`。
6. 遍历选择列表 (当前数字对应的字母):
 - 遍历 `letters` 中的每个字母 `letter`。
 - 做选择: 将 `letter` 追加到 `current_combination` (如果是字符串, 则 `current_combination + letter`)。
 - 递归进入下一层决策: 调用 `backtrack(index + 1, current_combination + letter)`。
 - 撤销选择 (回溯): 在回溯函数中, 由于字符串是不可变的, 每次递归传递的是新的字符串, 所以不需要显式地撤销选择。如果 `current_combination` 使用列表实现, 则需要在递归返回后 `pop()`。
7. 主函数调用:
 - 初始化结果列表 `result = []`。
 - 调用 `backtrack(0, "")` 开始回溯 (初始索引为 0, 初始组合为空字符串)。
 - 返回 `result`。

3. 代码实现 (Python)

```
from typing import List

class Solution:
    def letterCombinations(self, digits: str) -> List[str]:
        """
        使用回溯算法生成电话号码的字母组合。
        Args:
            digits: 仅包含数字 '2'-'9' 的字符串。
        Returns:
            所有可能的字母组合列表。
        """
        # 处理空输入
        if not digits:
            return []

        # 数字到字母的映射
```

```

mapping = {
    '2': "abc", '3': "def", '4': "ghi", '5': "jkl",
    '6': "mno", '7': "pqrs", '8': "tuv", '9': "wxyz"
}

result = [] # 存储最终结果

def backtrack(index: int, current_combination: str):
    """
    回溯函数核心逻辑。
    Args:
        index: 当前处理的 digits 字符串的索引。
        current_combination: 当前构建的字母组合字符串。
    """
    # 递归终止条件: 当索引到达 digits 末尾时, 找到一个完整组合
    if index == len(digits):
        result.append(current_combination)
        return

    # 获取当前数字和对应的字母
    current_digit = digits[index]
    letters = mapping[current_digit]

    # 遍历当前数字对应的所有字母
    for letter in letters:
        # 做选择 (隐式地通过传递新字符串完成)
        # 递归进入下一层决策, 处理下一个数字
        backtrack(index + 1, current_combination + letter)
        # 撤销选择 (隐式地在递归返回时完成, 因为字符串不可变)

    # 从索引 0 和空字符串开始回溯
    backtrack(0, "")
    return result

# 示例调用
# sol = Solution()
# print(sol.letterCombinations("23"))
# print(sol.letterCombinations(""))
# print(sol.letterCombinations("2"))

```

4. 复杂度分析

设输入的数字字符串 *digits* 的长度为 *N*, 每个数字平均对应 *M* 个字母 (*M* 在 3 到 4 之间)。

- 时间复杂度: $O(M^N * N)$ 或 $O(M^N)$

- 状态树的叶子节点数量（即最终组合的数量）大约是 M^N 。
- 到达每个叶子节点需要 N 步。
- 如果将字符串拼接视为 $O(1)$ （在某些语言实现中可能接近），则时间复杂度为 $O(M^N)$ 。
- 如果字符串拼接视为 $O(\text{当前长度})$ ，则每次拼接成本增加，总时间复杂度更接近 $O(N * M^N)$ ，因为每个最终组合长度为 N ，构建它需要大约 N 次拼接操作。在 Python 中，字符串拼接通常会创建新字符串，成本较高。
- 更精确地说，状态树的总节点数约为 $(M^{(N+1)} - 1) / (M - 1)$ ，每个节点处理时间为 $O(M)$ （遍历字母），所以是 $O(M * M^N) = O(M^{(N+1)})$ ？不太对。
- 考虑最终结果的数量是 M^N ，每个结果长度为 N 。生成所有结果的总操作数与结果的总字符数成正比，即 $O(N * M^N)$ 。
- 空间复杂度： $O(N)$
 - 递归调用栈的深度最多为 N 。
 - `current_combination` 字符串（或列表）在递归过程中最大长度为 N 。
 - 结果列表 `result` 需要存储 M^N 个长度为 N 的字符串，空间为 $O(N * M^N)$ ，但这通常不计入算法本身的额外空间复杂度。

5. 如何在面试中讲解

1. 问题理解与映射：
 - “目标是根据电话按键的数字字母映射，生成给定数字字符串的所有可能字母组合。”
 - “首先，我们需要一个映射表来存储 ‘2’ 到 ‘9’ 对应的字母。”
2. 核心思路：回溯：
 - “这个问题可以看作是多层决策。每一层对应输入数字字符串中的一个数字，我们需要在这一层选择该数字对应的一个字母。”
 - “使用回溯算法，通过 DFS 探索所有可能的字母选择路径。”
3. 回溯过程：
 - “定义一个递归函数 `backtrack(index, currentString)`，`index` 是当前处理的数字索引，`currentString` 是已构建的字母组合。”
 - “结束条件：当 `index` 等于数字字符串的长度时，表示我们已经为每个数字选择了一个字母，`currentString` 是一个完整组合，将其加入结果列表。”
 - “选择列表：获取 `digits[index]` 对应的字母列表 `letters`。”
 - “递归：遍历 `letters` 中的每个 `letter`。将 `letter` 追加到 `currentString`，然后递归调用 `backtrack(index + 1, newString)` 来处理下一个数字。”
 - “撤销选择：由于 Python 字符串是不可变的，每次递归传递的是新字符串 `currentString + letter`，所以回溯（状态恢复）是自动发生的，不需要显式操作。如果用列表来构建组合，则需要在递归返回后 `pop()`。”
4. 初始调用与边界：
 - “主函数需要处理输入为空的情况（返回空列表）。”
 - “初始化结果列表，然后调用 `backtrack(0, "")` 开始递归。”
5. 复杂度分析：

- “设数字串长度为 N ，平均每个数字对应 M 个字母。”
- “时间复杂度大致是 $O(N * M^N)$ ，因为大约有 M^N 个最终组合，每个组合长度为 N ，生成它们需要相应的时间。”
- “空间复杂度是 $O(N)$ ，主要是递归栈的深度。”